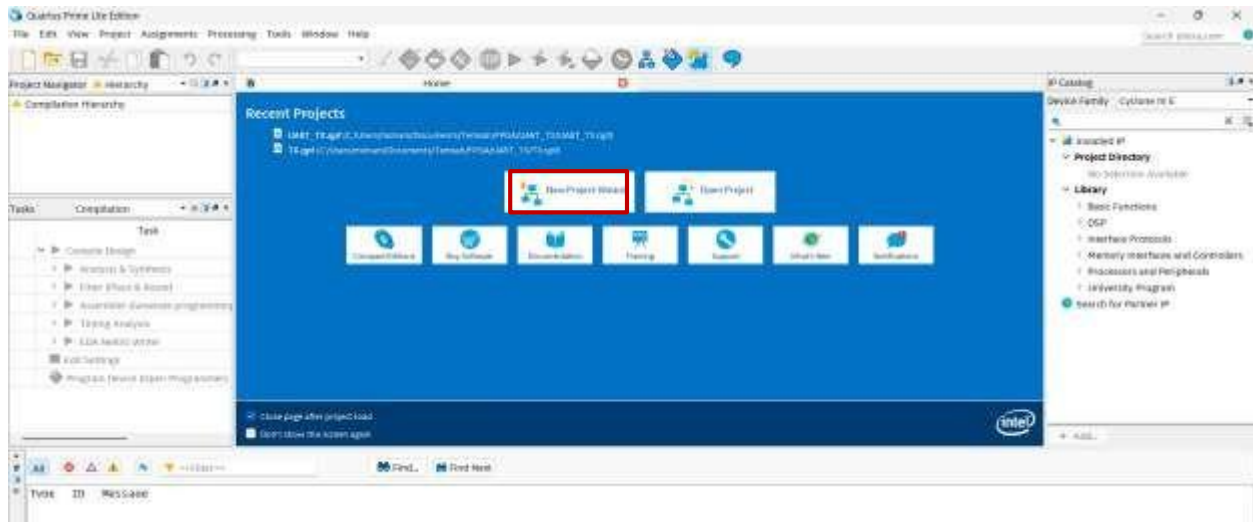


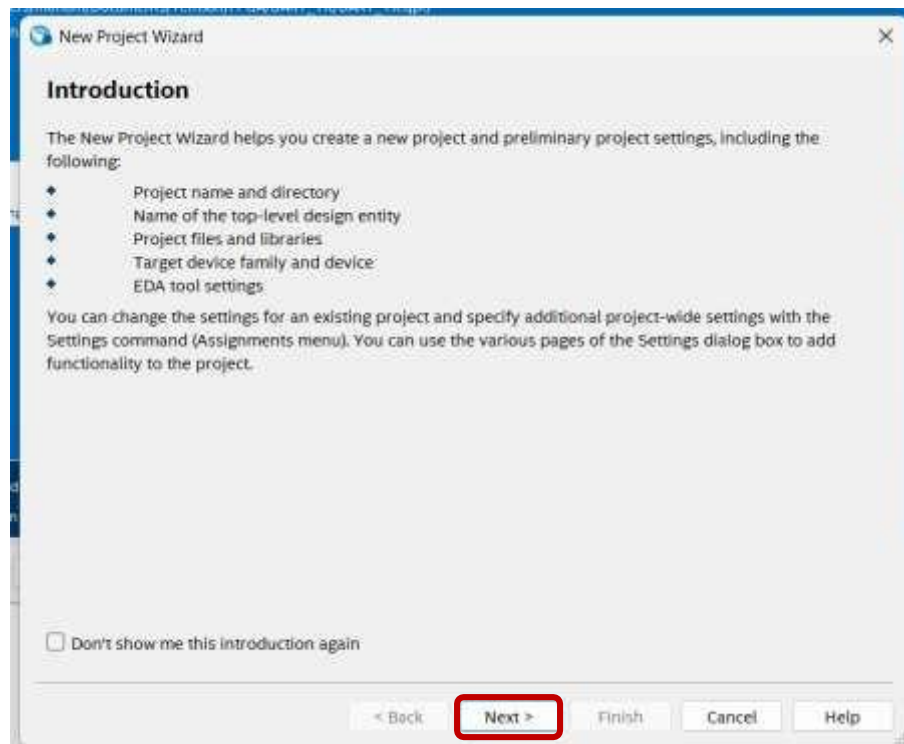
Lab2.2

Step 1: Project setup

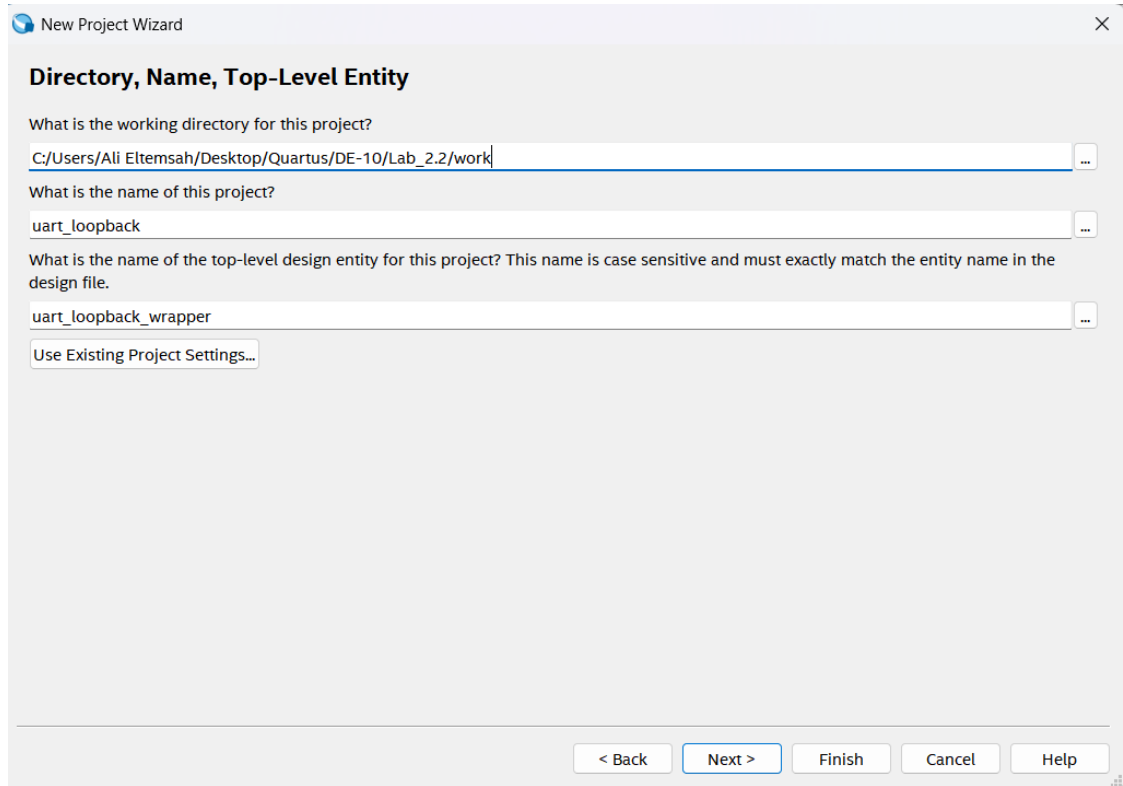
- Launch **Intel Quartus Prime Lite Edition** and click on **New Project Wizard** on the home landing page.
- Initializes a new design workspace, where the project directory, top-level entity name (**top_module**), target FPGA family (**MAX 10** or **Cyclone IV**), and EDA tools are defined.



- Click **Next >** on the wizard introduction dialog to proceed with project configuration.

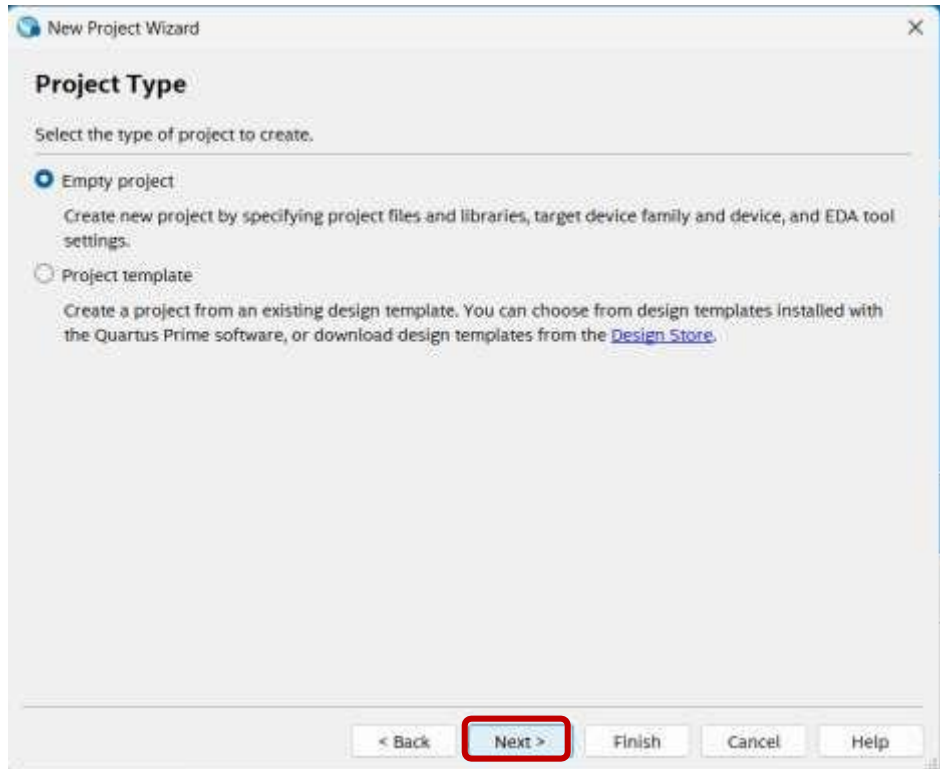


- Set the working directory path, set the project name to **LFSR**, and set the top-level entity name to **LFSR**. Click **Next >**.



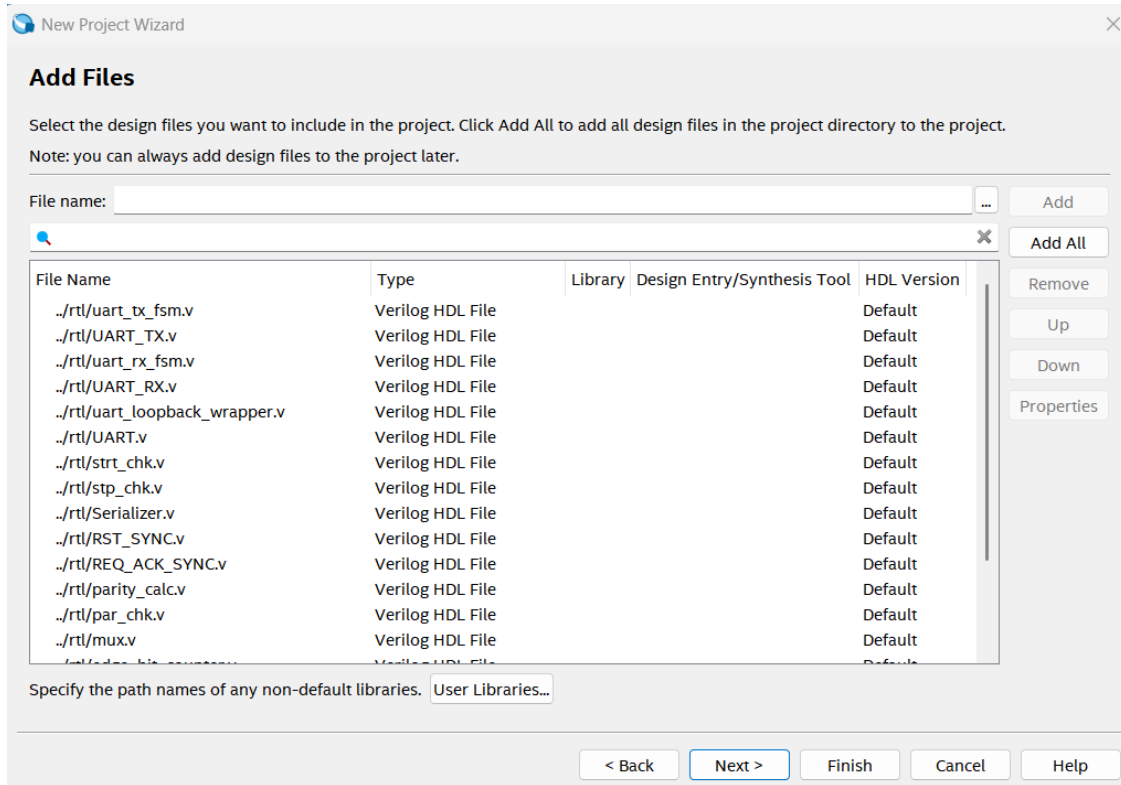
The screenshot shows the 'New Project Wizard' window with the title 'New Project Wizard'. The main heading is 'Directory, Name, Top-Level Entity'. There are three text input fields with '...' buttons to their right. The first field is labeled 'What is the working directory for this project?' and contains the path 'C:/Users/Ali Eltemsah/Desktop/Quartus/DE-10/Lab_2.2/work'. The second field is labeled 'What is the name of this project?' and contains 'uart_loopback'. The third field is labeled 'What is the name of the top-level design entity for this project? This name is case sensitive and must exactly match the entity name in the design file.' and contains 'uart_loopback_wrapper'. Below these fields is a button labeled 'Use Existing Project Settings...'. At the bottom right, there are five buttons: '< Back', 'Next >', 'Finish', 'Cancel', and 'Help'. The 'Next >' button is highlighted with a blue border.

- Select **Empty project** and click **Next >** to create a fresh workspace for importing HDL source files and IP blocks.
-

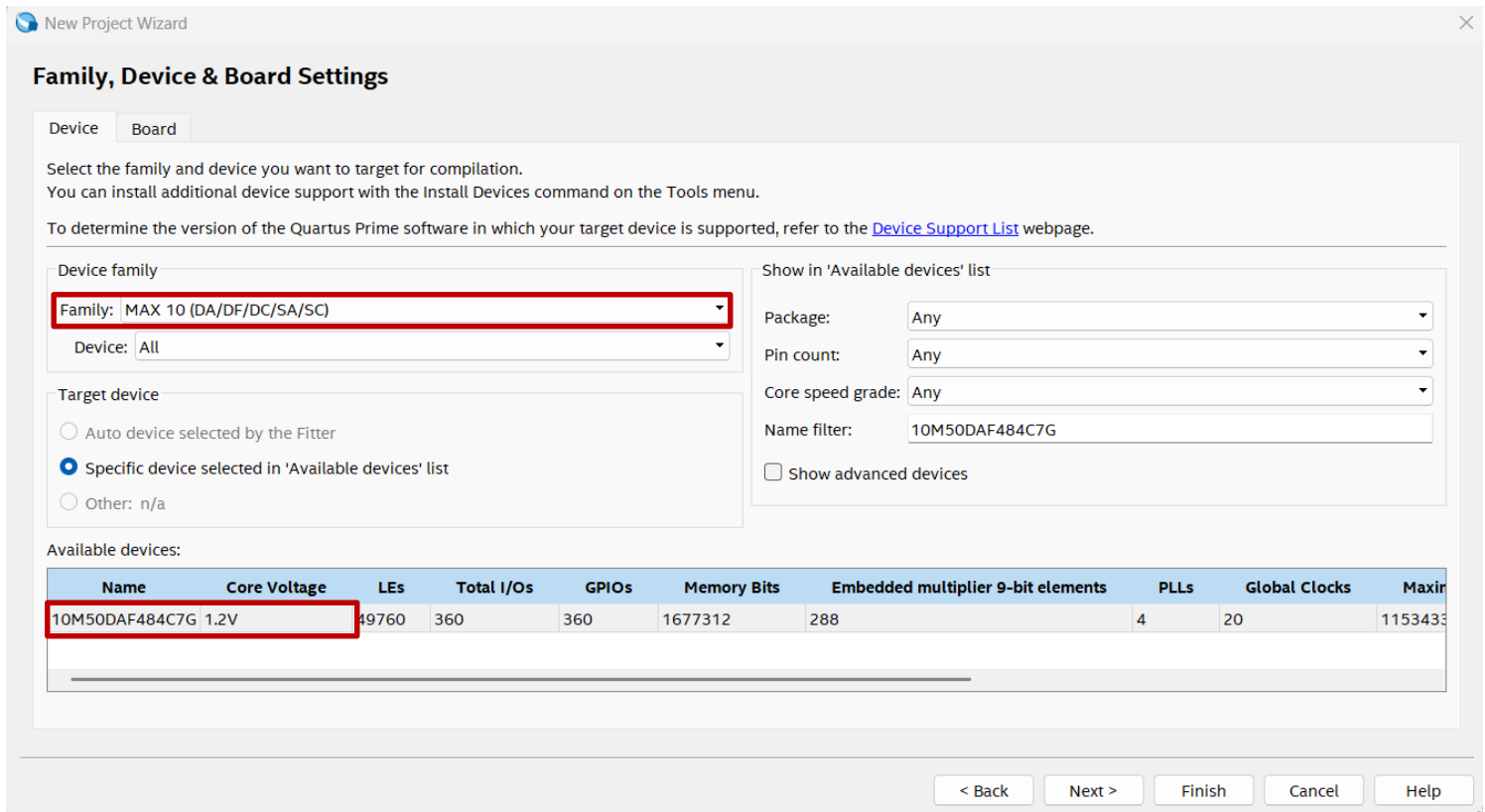


The screenshot shows the 'New Project Wizard' window with the title 'New Project Wizard'. The main heading is 'Project Type'. Below the heading is the text 'Select the type of project to create.' There are two radio button options. The first option is 'Empty project', which is selected (indicated by a blue dot). Below it is the text 'Create new project by specifying project files and libraries, target device family and device, and EDA tool settings.' The second option is 'Project template', which is not selected. Below it is the text 'Create a project from an existing design template. You can choose from design templates installed with the Quartus Prime software, or download design templates from the [Design Store](#).' At the bottom right, there are five buttons: '< Back', 'Next >', 'Finish', 'Cancel', and 'Help'. The 'Next >' button is highlighted with a red rectangle.

- Add all Verilog design files (such as **LFSR.v**) to the project by clicking **Add All**, then click **Next >**.

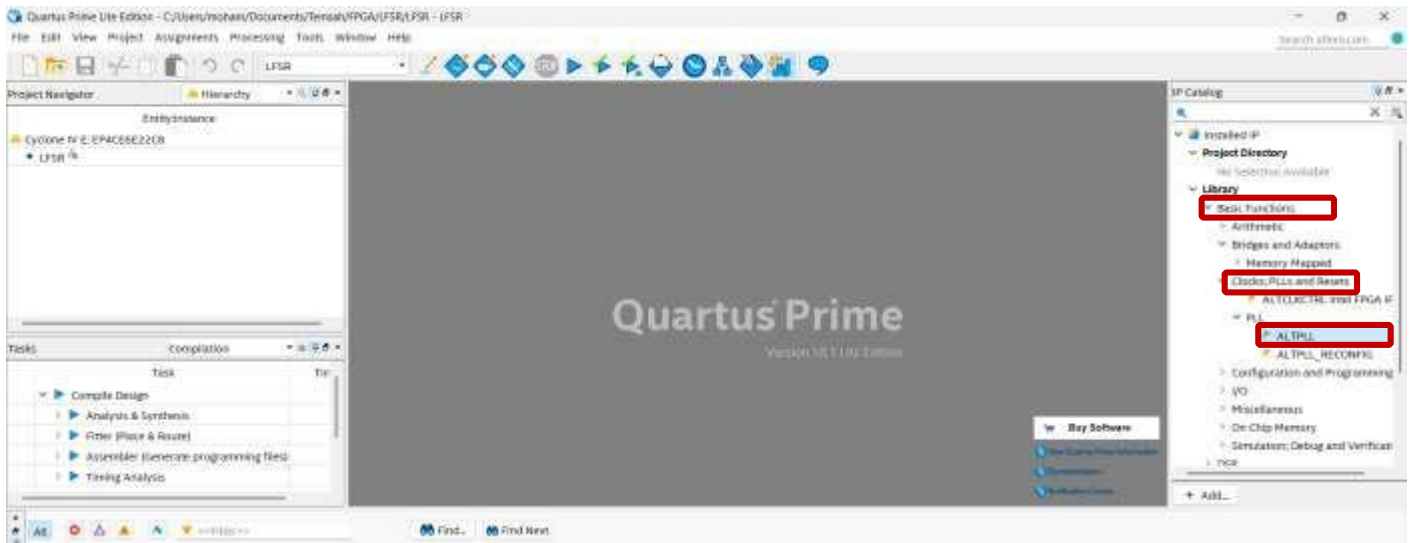


- Set the Device family to **MAX 10**, select the target FPGA device **10M50DAF484C7G**, and click **Finish** to instantiate the workspace.

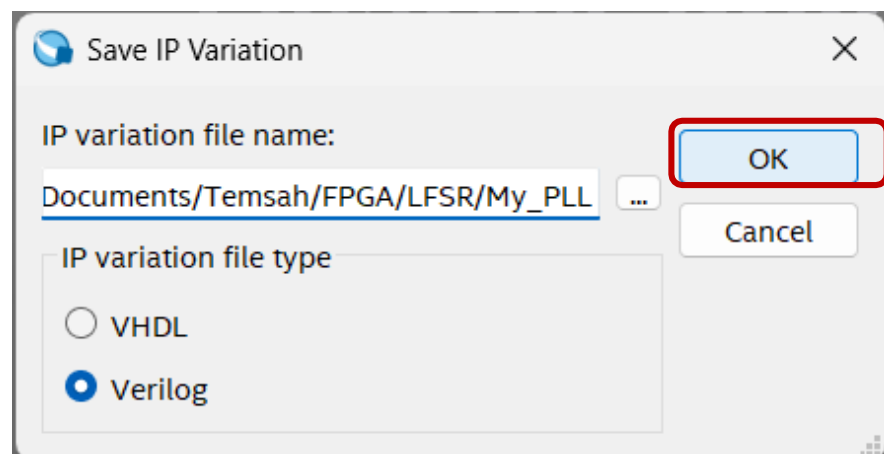


Step 2: Clock Management

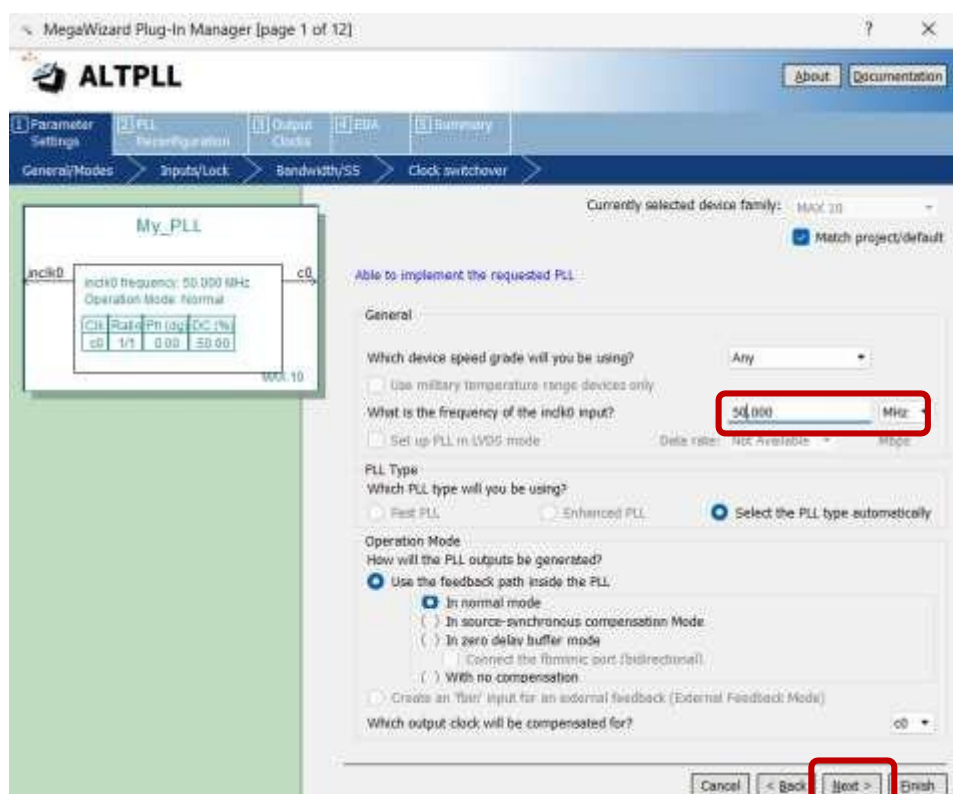
- In the **IP Catalog** panel, expand **Library** -> **Basic Functions** -> **Clocks, PLLs and Resets** -> **PLL**, and double-click **ALTPLL**.



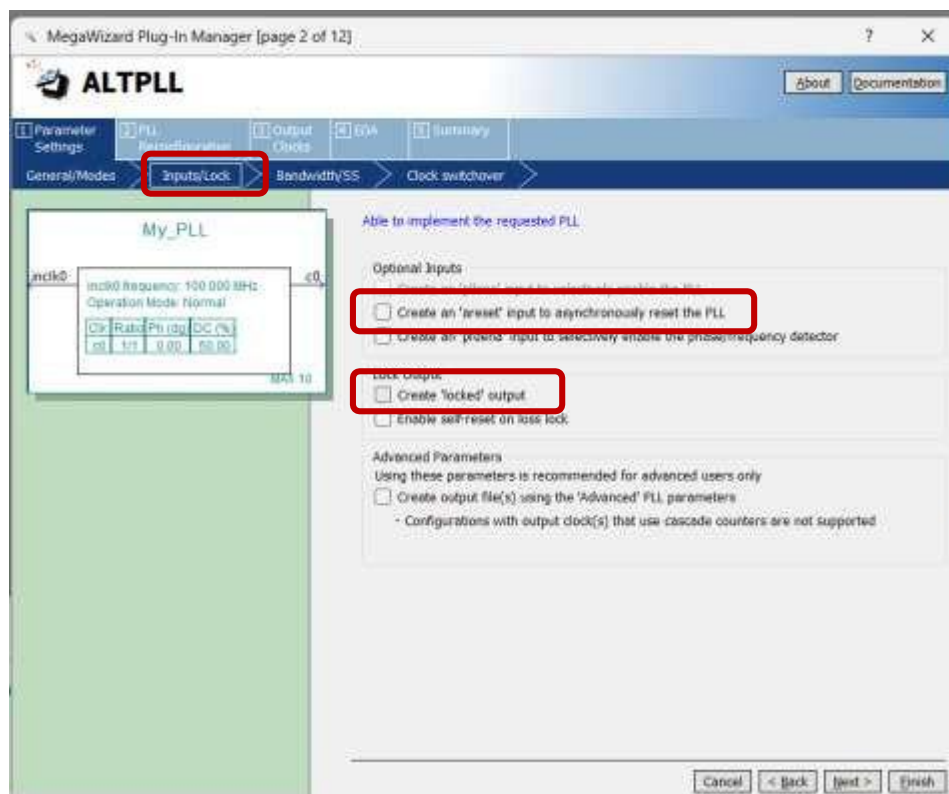
- Set the IP variation file name to **My_PLL**, select **Verilog** as the IP variation file type, and click **OK**.



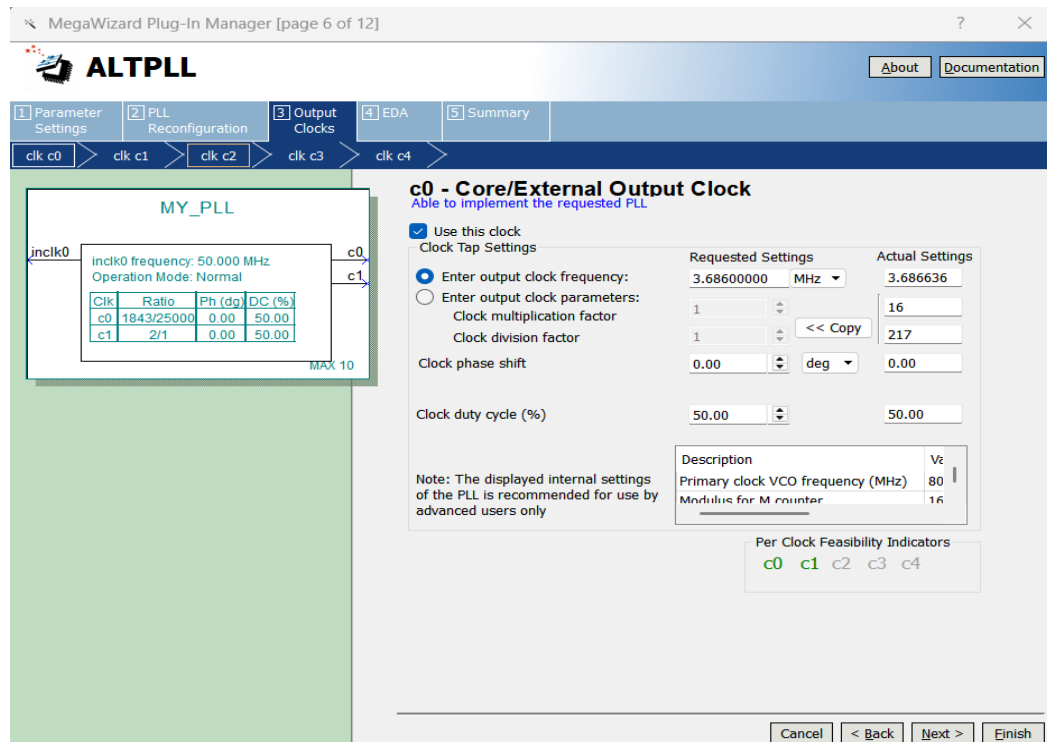
- On the **General/Modes** tab, set the inclk0 input clock frequency to **50.000 MHz** and click **Next >**.



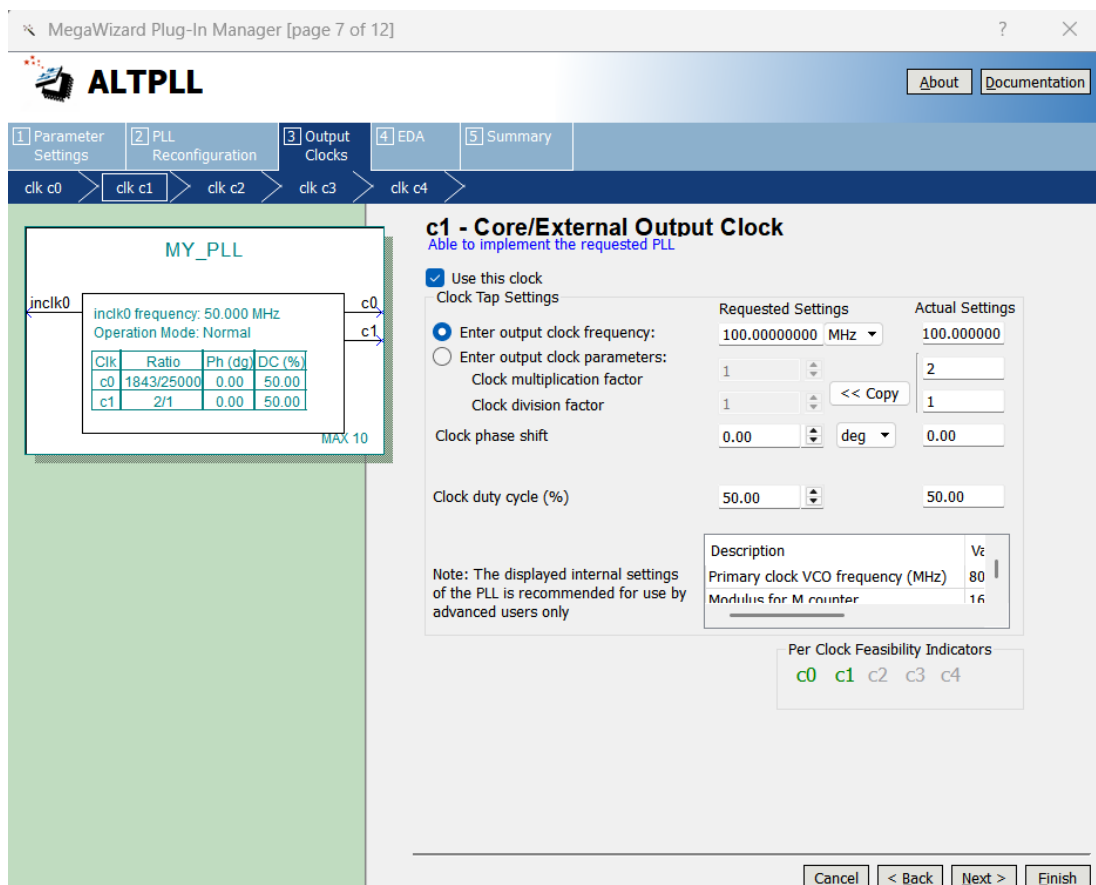
- On the **Inputs/Lock** tab, uncheck 'areset' (asynchronous reset) and 'locked' (lock output) to simplify the PLL clock interface, then click **Next >**.



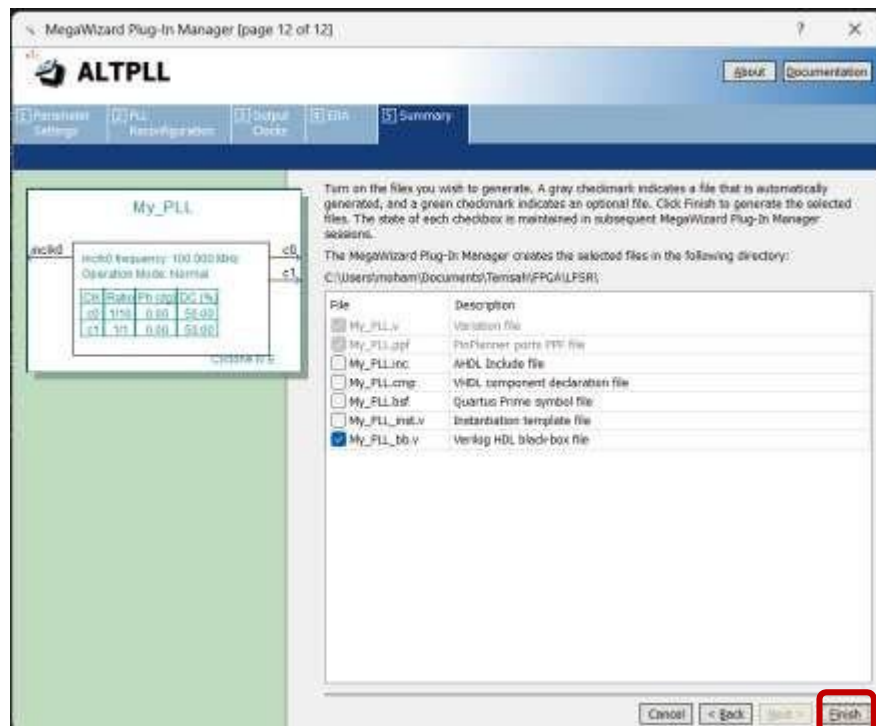
- On the **clk c0** tab, select **Enter output clock frequency** and set the frequency to **3.686 MHz** for system logic execution.



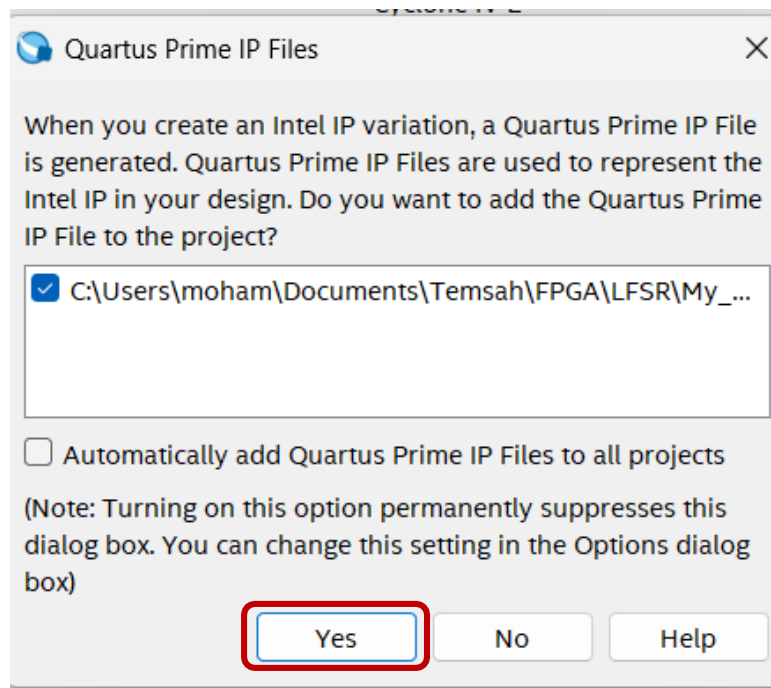
- On the **clk c1** tab, select **Enter output clock frequency** and set the frequency to **100.000 MHz** for Signal Tap oversampling.



- Navigate to the **Summary** tab, ensure the target IP output files are checked, and click **Finish** to generate the ALTPLL core.

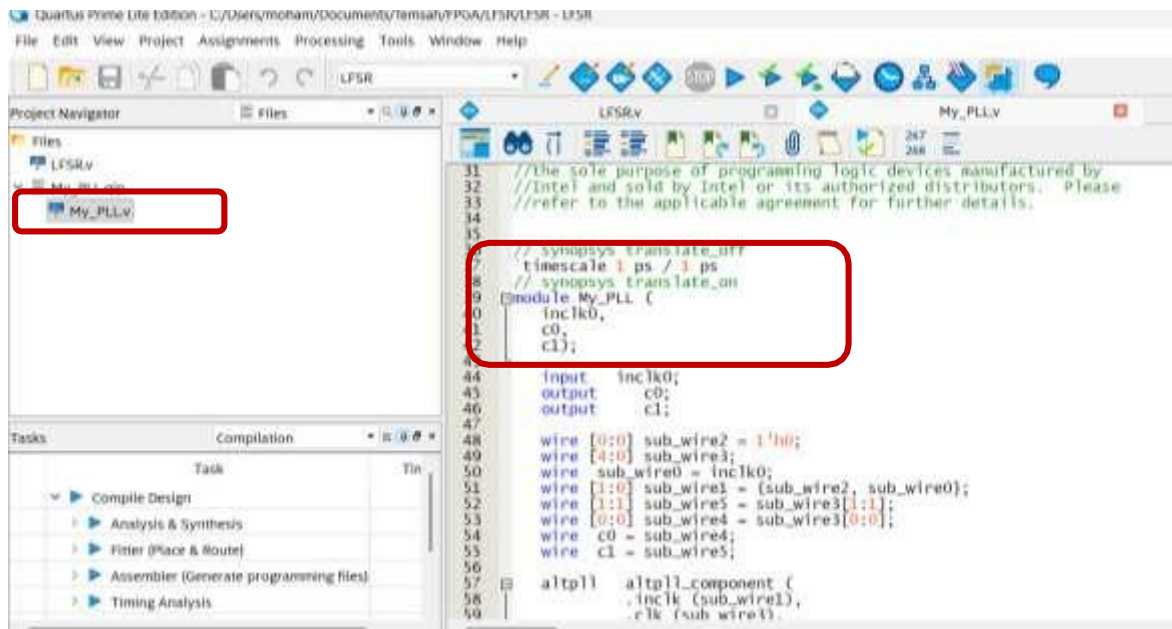


- Click **Yes** on the pop-up dialog to automatically add the generated IP variation file (**MY_PLL.qip**) to the Quartus project.



- Double-click **My_PLL.v** in the Project Navigator to inspect the generated module interface (incl0, c0, c1)

- Instantiate the **My_PLL** IP block inside the top-level wrapper (**top.v**) to route the 50 MHz input clock to the internal logic clock (c0) and Signal Tap sampling clock (c1).



```

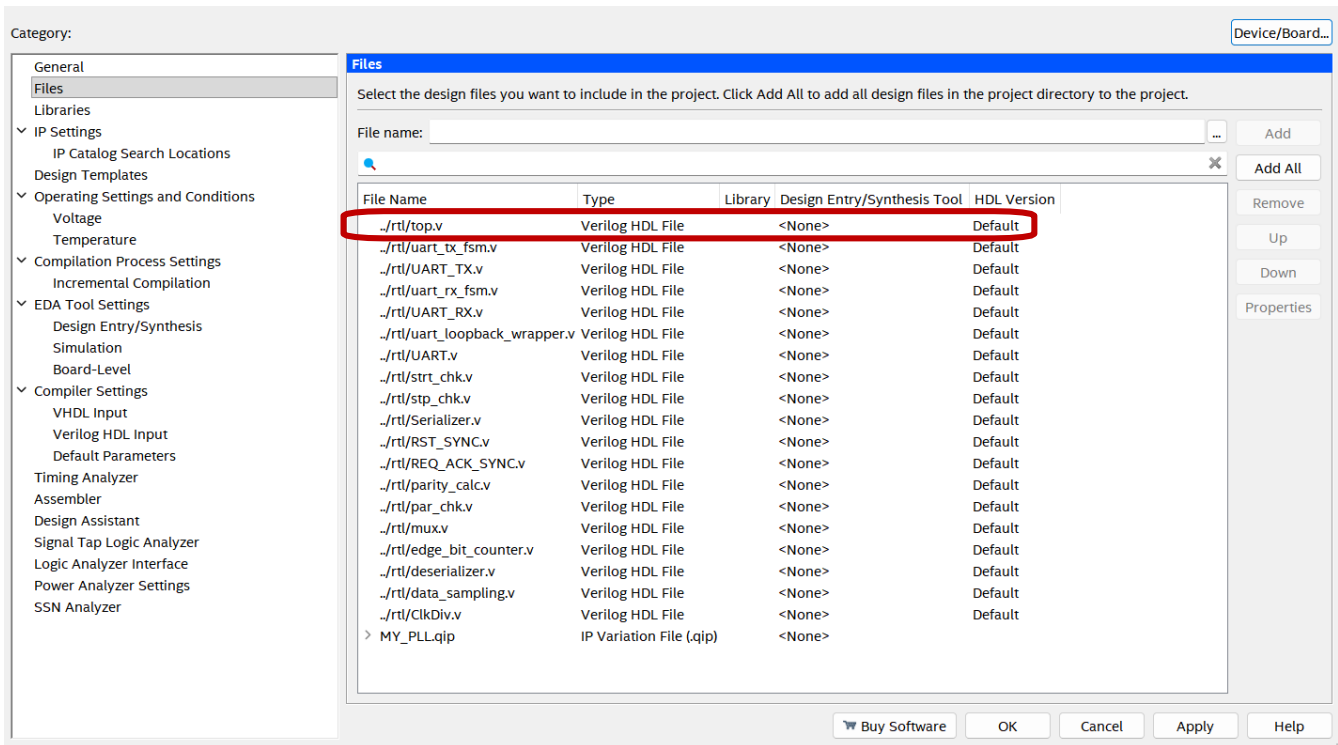
module top (
    input wire    CLOCK_50,    // 50 MHz board oscillator
    input wire    RST_N,      // KEY[0] active-low reset
    // Serial interface
    input wire    uart_rx,    // from peer TX
    output wire   uart_tx,    // to peer RX
    // Status / control
    output wire   TX_OUT_V,   // TX busy
    output wire   parity_error,
    output wire   framing_error
);

// -----
// PLL: 50 MHz -> 3.684 MHz (rx_clk). tx_clk (baud) is generated inside
// uart_loopback_wrapper as rx_clk / 32.
// -----
wire    rx_clk;
wire tap_clk; // 36.8 MHz SignalTap sampling clock (PLL c1)

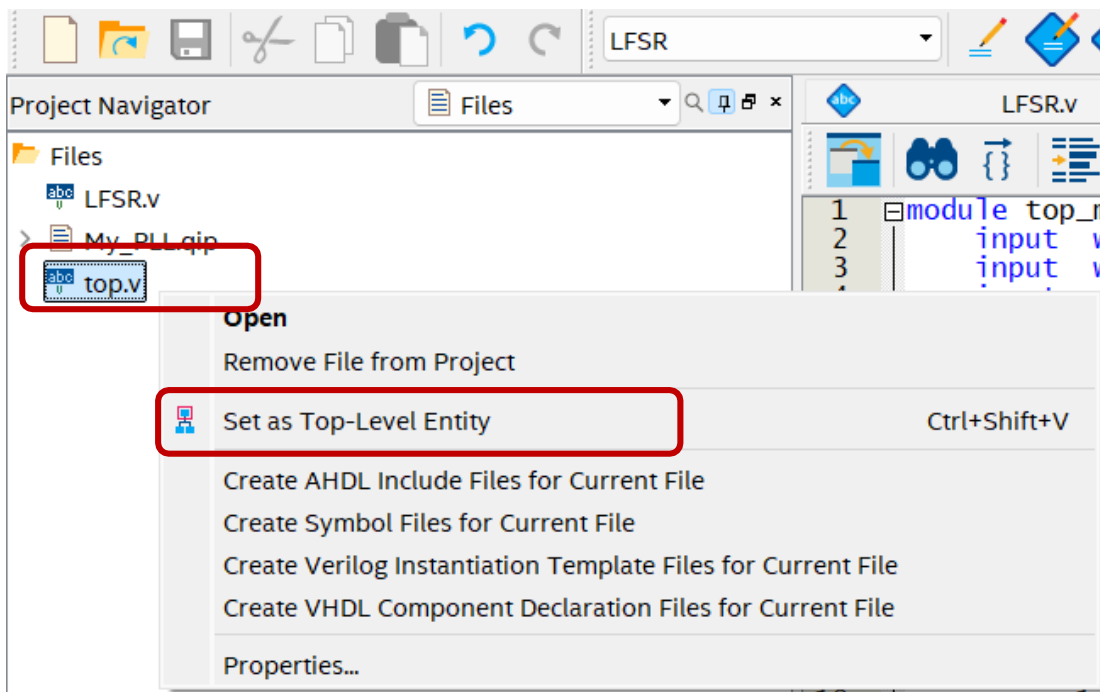
My_PLL U_PLL (
    .inc1k0 (CLOCK_50),
    .c0     (rx_clk),
    .c1     (tap_clk)
);

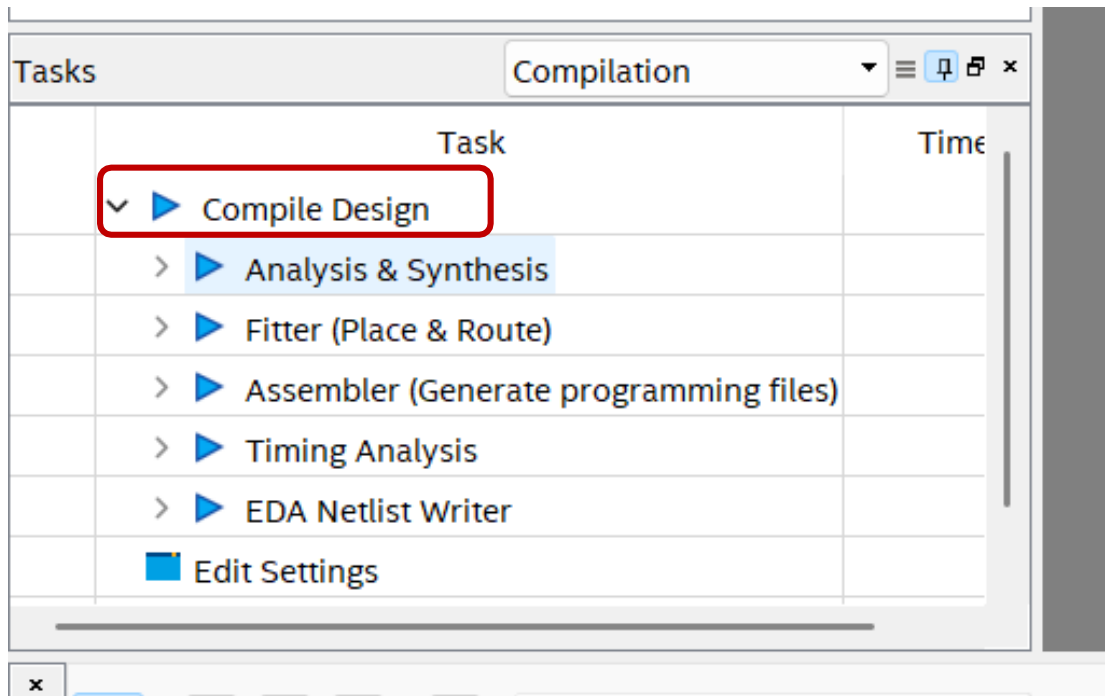
```

- Add **top.v** in the project RTL files

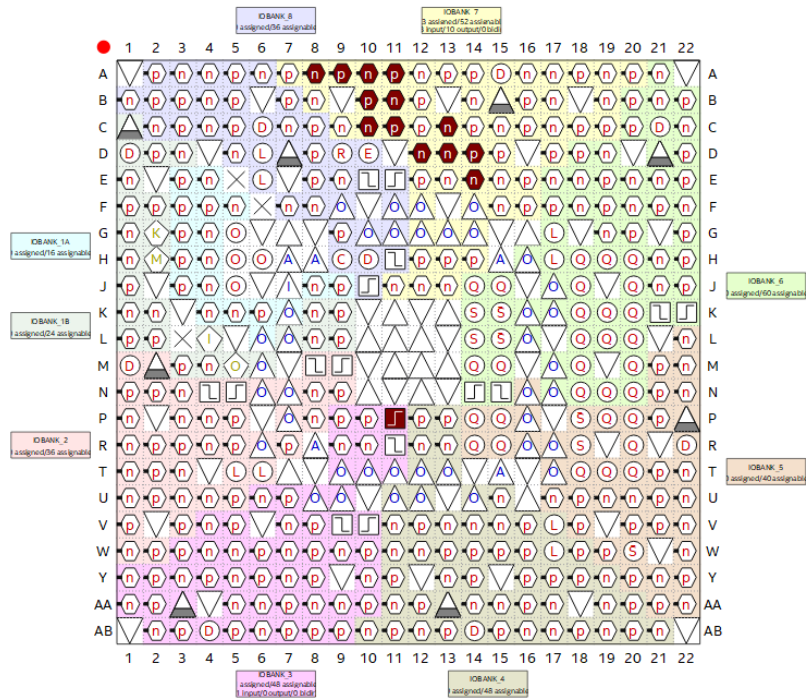


- Right-click **top.v** in the Project Navigator and select **Set as Top-Level Entity**.
- In the Tasks pane on the left, double-click **Analysis & Synthesis** to compile and verify the RTL design structure.



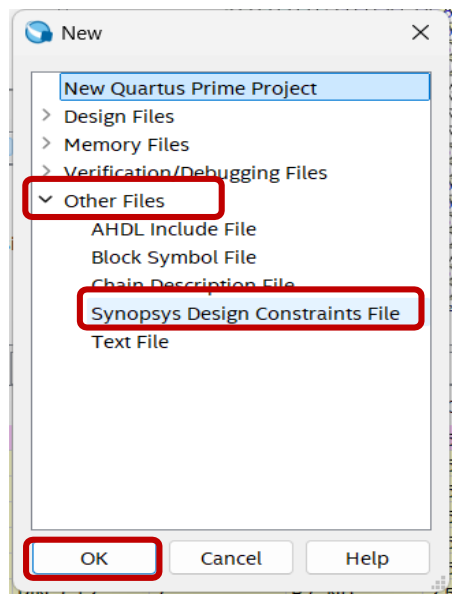


- Open **Pin Planner** by clicking the Pin Planner icon on the top toolbar (or **Ctrl + Shift + N**).
- Assign the physical target FPGA pin locations for clk_50m, rst_n, LFSR_EN, OUT_EN, LEDs[7:0], valid_out, and crc_out.

[illegible]

Step 4: Timing Constraints

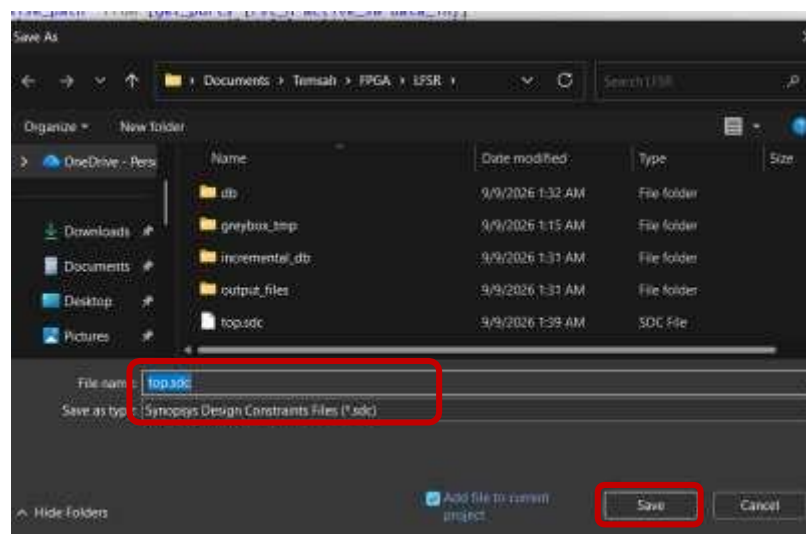
- Navigate to File -> New, expand Other Files, select Synopsys Design Constraints File, and click OK.



- Enter the SDC timing constraints (create_clock, derive_pll_clocks, set_false_path) into the editor.

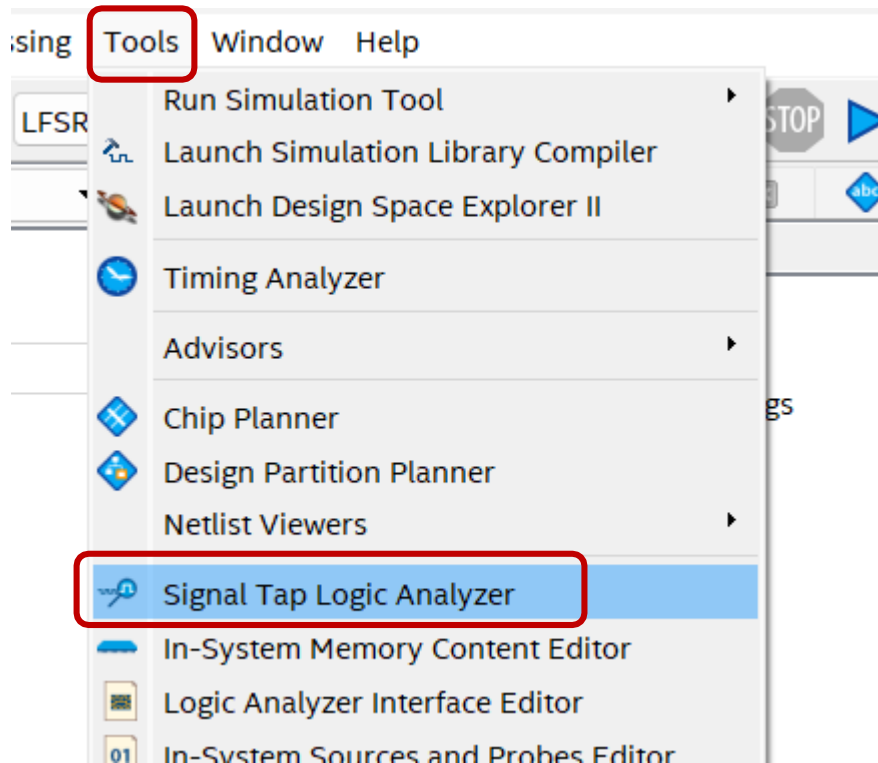
```
create_clock -name clk_50m -period 20.000 [get_ports clk_50m]
derive_pll_clocks
derive_clock_uncertainty
set_input_delay 0.5 -clock clk_50m [get_ports {LFSR_EN OUT_EN data_in}]
set_output_delay 0.5 -clock clk_50m [get_ports {LEDs valid_out crc_out}]
```

- Save the constraint file as top.sdc in the root project directory.

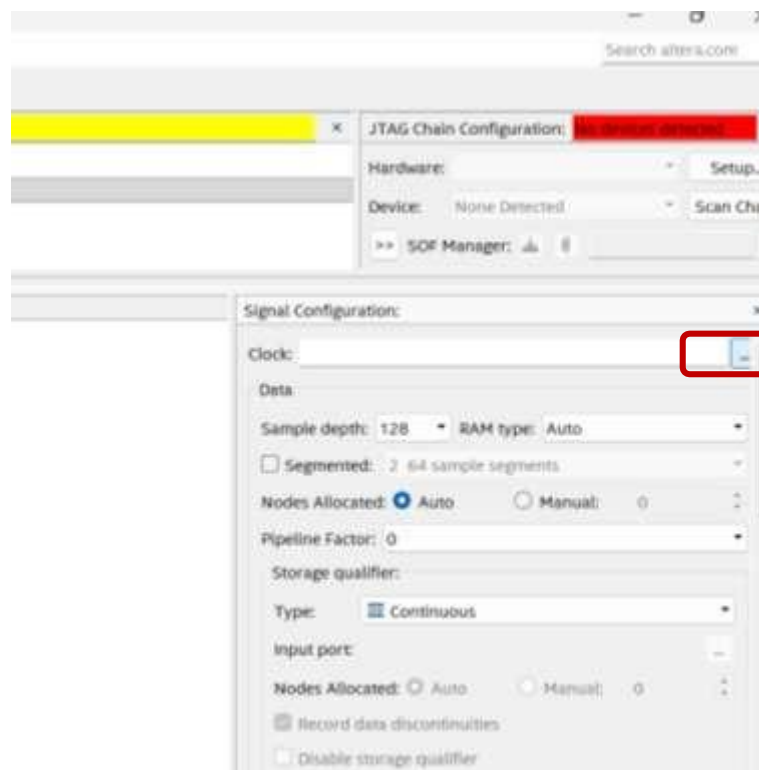


Step 5: Logic Analyzer

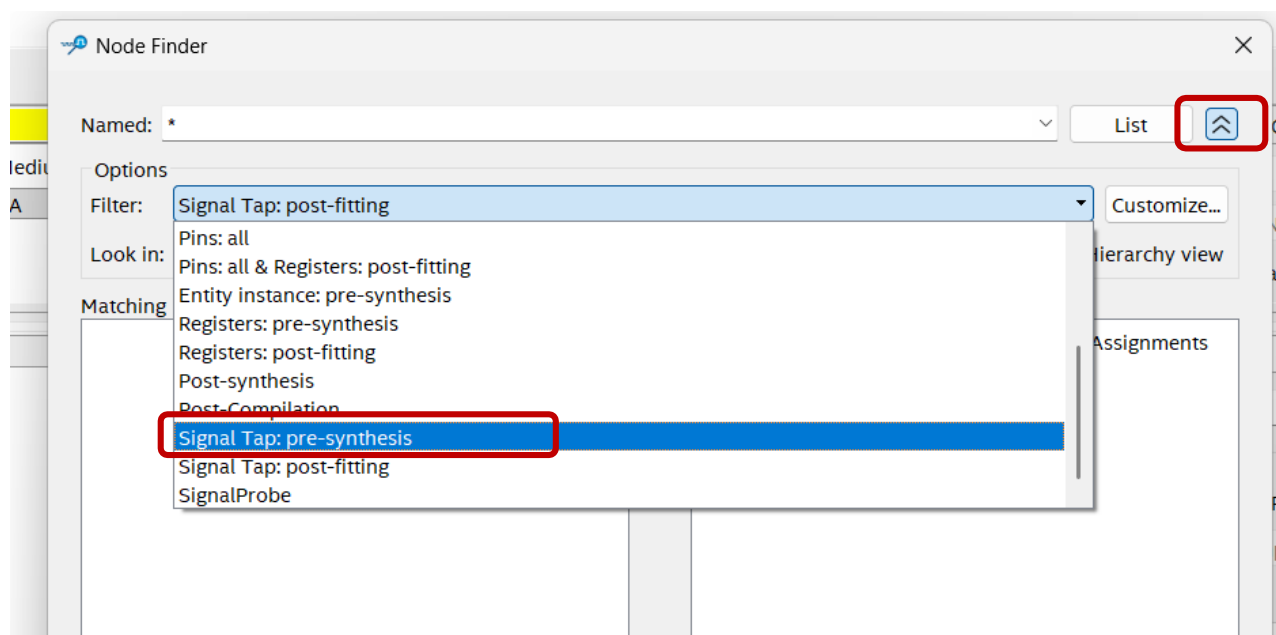
- Launch the tool by selecting **Tools -> Signal Tap Logic Analyzer** from the top menu bar.



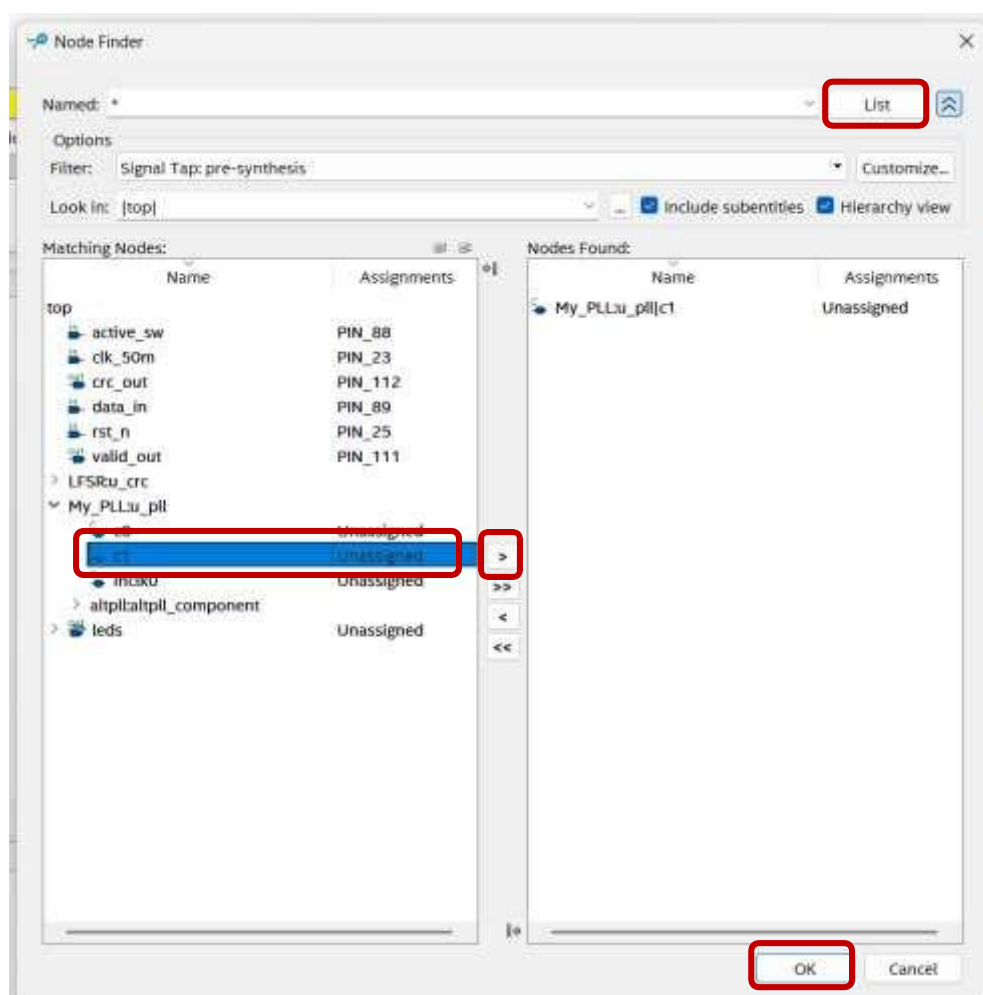
- In the **Signal Configuration** panel on the right, click the ... button next to **Clock** to assign the sampling clock.



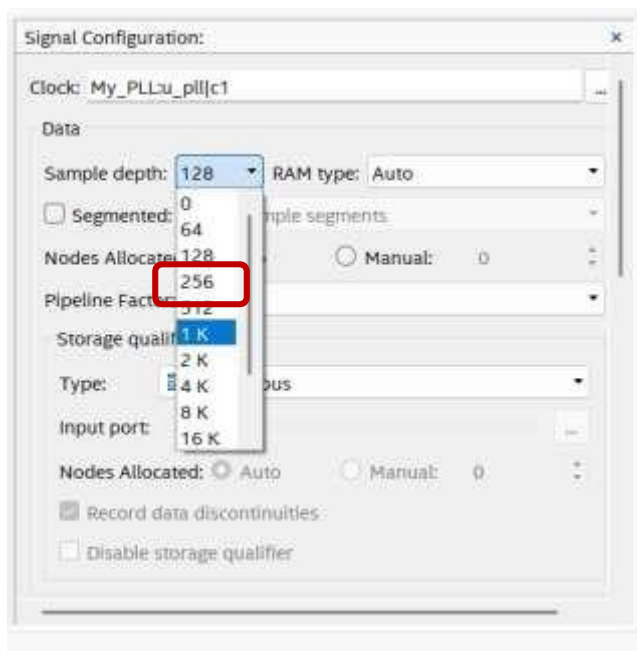
- In the Node Finder window, set the **Filter** to **Signal Tap: pre-synthesis** and click **List**.



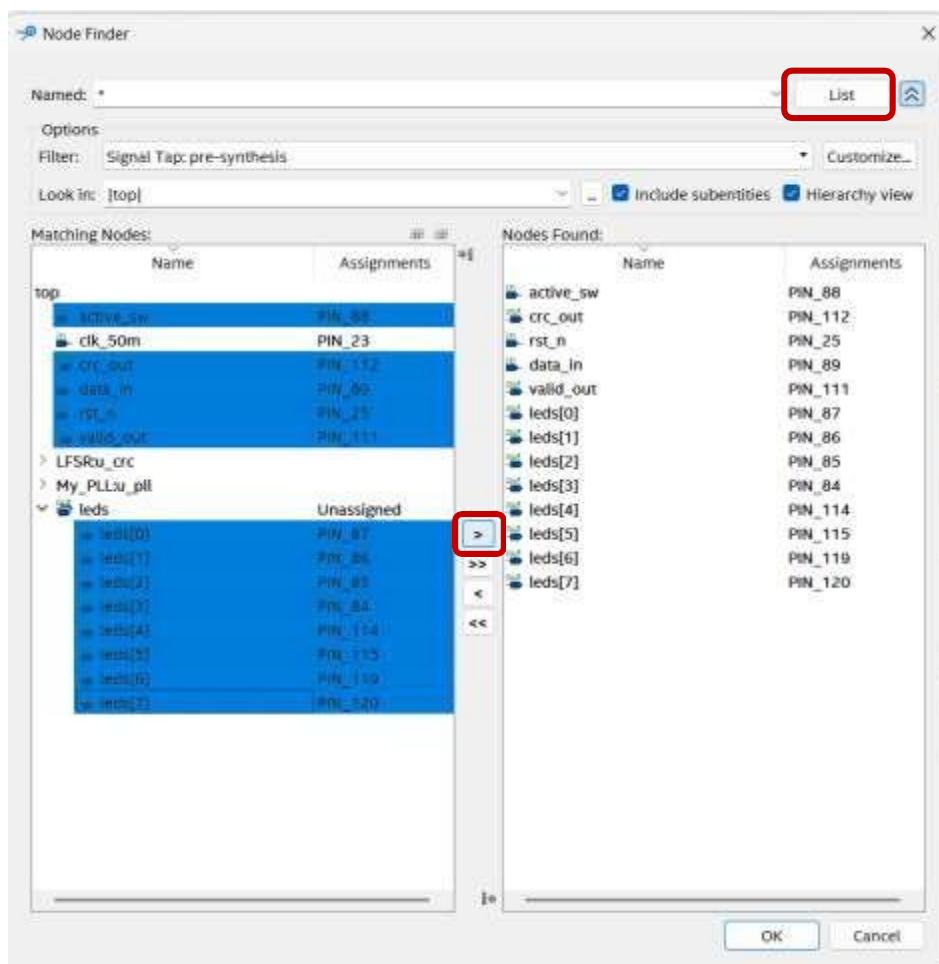
- Select the fast PLL clock output (**My_PLL:u_pll|c1**), click > to add it to **Nodes Found**, and click **OK**.



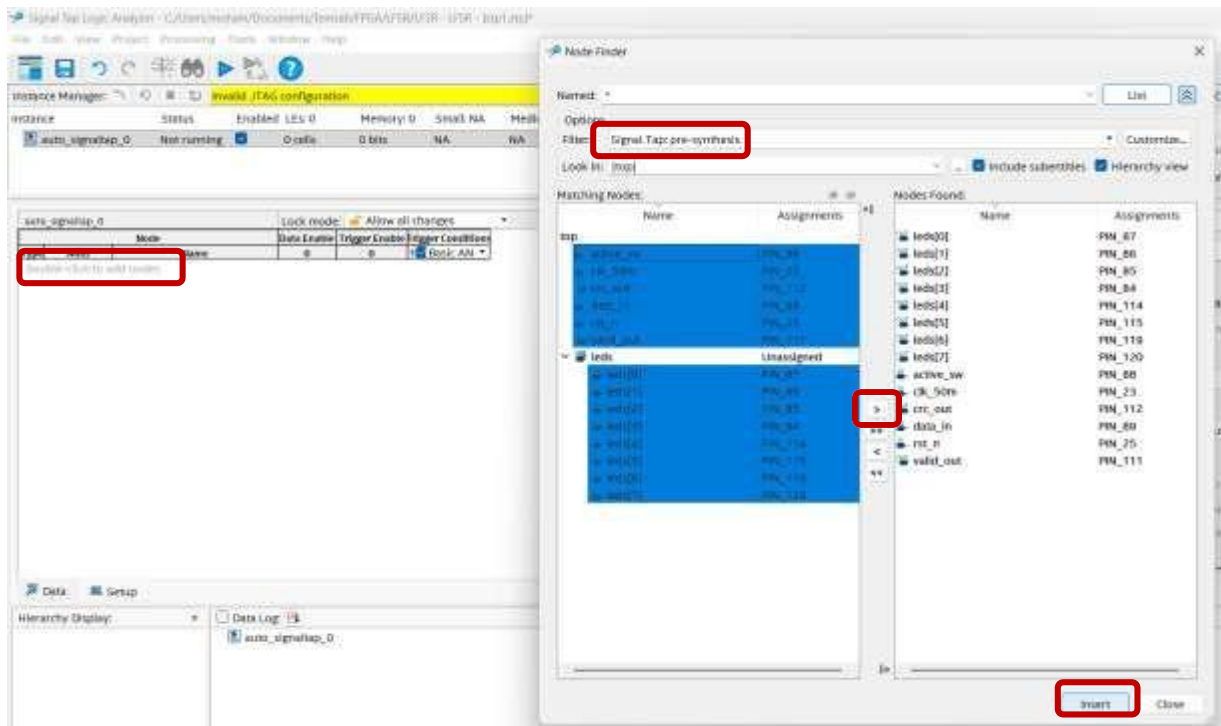
- Change the **Sample depth** setting to **1 K** (1024 samples) in the Signal Configuration pane.



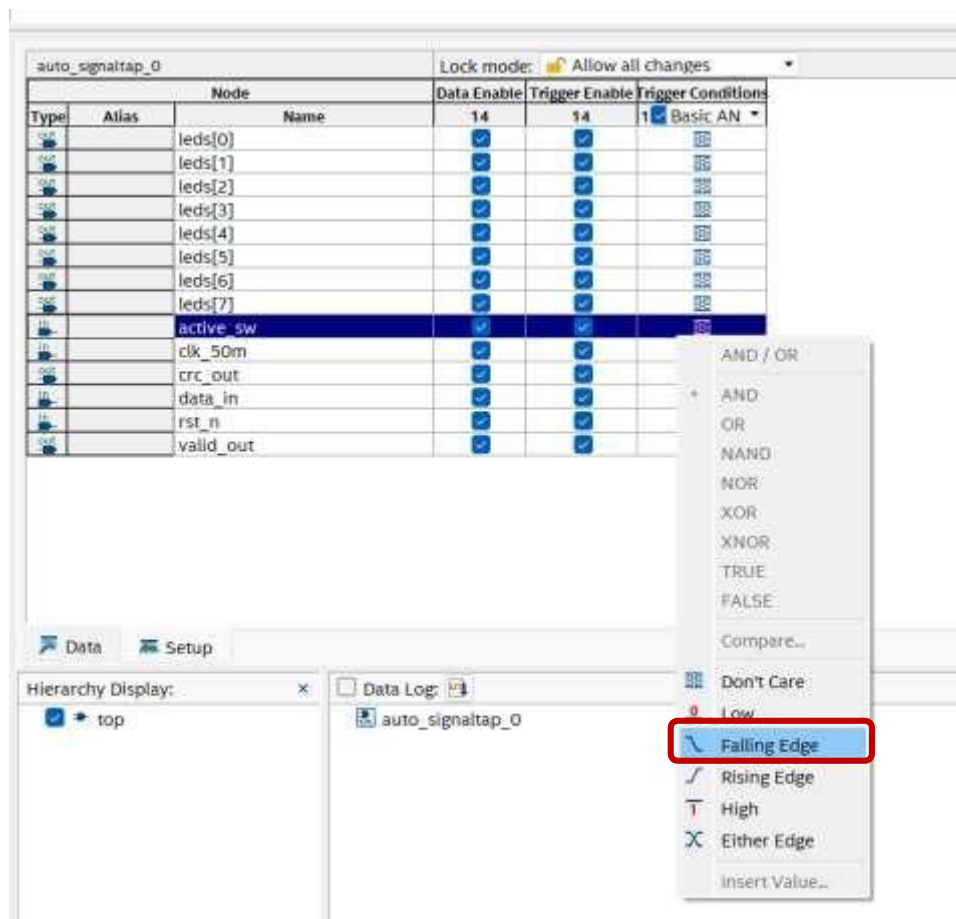
- Double-click the Setup table to open Node Finder, select all project signals (active_sw, crc_out, data_in, rst_n, valid_out, leds[7:0]), and click > to add them.



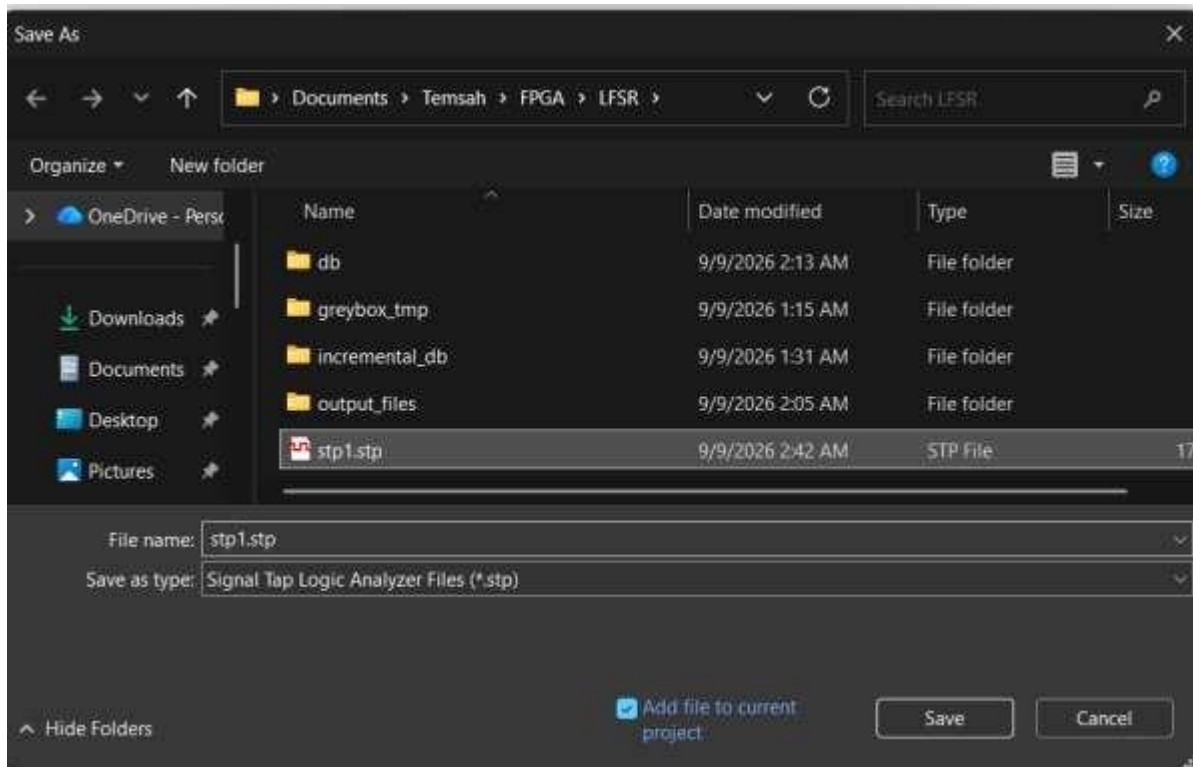
- Click **OK** to insert all 13 signals into the Signal Tap setup grid.



- Right-click the **Trigger Conditions** cell for **active_sw** and select **Falling Edge (\)** to set the trigger.



- Save the file as **stp1.stp** in the project directory and click **Yes** to enable Signal Tap in the active project.



- In the Tasks pane on the left, double-click **Compile Design** to run full synthesis and embed Signal Tap into the hardware bitstream.

Tasks			Compilation			
		Task				Time
✓	▼	▶ Compile Design				00:00:48
✓	>	▶ Analysis & Synthesis				00:00:36
✓	>	▶ Fitter (Place & Route)				00:00:06
✓	>	▶ Assembler (Generate programming files)				00:00:02
✓	>	▶ Timing Analysis				00:00:04
	>	▶ EDA Netlist Writer				
		■ Edit Settings				
		■ Program Device (Open Programmer)				